

Indian Institute of Technology, Ropar

20 August, 2026



EE208 Control Engineering Lab Report

Experiment 1

Group no.: 11A

(Saurav - 2024EPB1279

Ashwin - 2024EPB1262

Sumitha S - 2022EPB1251)

1. Objective

The standard formulae for percentage overshoot, peak time, settling time, and steady-state error are derived for an ideal second-order system with no finite zeros. In practice, higher-order systems with additional poles and zeros can behave differently. This experiment examines how accurately these second-order formulae describe a fourth-order feedback system with two finite zeros, and under what conditions the approximation becomes reliable.

The fourth-order open-loop plant is connected in unity negative feedback, and the proportional gain (K) is varied over a wide range. For eight representative gain values, the unit-step response is obtained from simulation, and the four performance measures are calculated directly from the response. These simulated values are taken as the measured results. The same quantities are then estimated using the natural frequency (ω_n) and damping ratio (ζ) obtained from the closed-loop pole pair closest to the imaginary axis. The measured and second-order estimates are compared to determine where the approximation becomes inaccurate and how the additional poles and zeros affect the response.

As specified in the laboratory procedure, the open-loop pole locations are also varied by $\pm 20\%$ to examine the effect of parameter uncertainty. The resulting changes are evaluated directly from the time-domain step responses, and the conclusions are based on the observed simulation results.

2. Theoretical Background

2.1 The standard second-order model

A canonical underdamped second-order system has the transfer function

$$T(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$$

whose complex-conjugate poles sit at $s = -\sigma \pm j\omega_d$, with $\sigma = \zeta\omega_n$ (the decay rate) and $\omega_d = \omega_n\sqrt{1-\zeta^2}$ (the damped oscillation frequency). Four performance metrics follow directly from ζ and ω_n alone, and these are exactly the formulae MATLAB's toolbox (and this experiment's code) uses whenever it needs a "second-order estimate":

$$PO = 100 \cdot \exp(-\pi\zeta / \sqrt{1-\zeta^2}) \% \quad (\text{percentage overshoot})$$

$$t_p = \pi / \omega_d \quad (\text{time to first peak})$$

$$t_s \approx 4 / \sigma = 4 / (\zeta\omega_n) \quad (2\% \text{ settling time})$$

$$e_{ss} = \lim_{s \rightarrow 0} s \cdot [R(s) - T(s)R(s)] \quad (\text{steady-state error; depends on system Type})$$

The first three formulae assume the response is generated by exactly one complex pole pair and nothing else — no other poles, no zeros. The fourth depends on how many free integrators (poles at the origin) the open loop contains: a Type-0 system gives a finite, non-zero e_{ss} to a step; Type-1 gives

ess = 0 to a step but a finite error to a ramp; Type-2 and higher give ess = 0 to both step and ramp inputs.

2.2 When the approximation can and cannot be trusted

For a higher-order system, the second-order formulae remain useful only if one complex pole pair truly dominates the transient — the classical “dominant-pole” idea. Two conditions are usually cited together as the textbook rule of thumb:

- Separation: every other pole should lie considerably further from the imaginary axis than the dominant pair — commonly quoted as a real-part ratio of five to ten times — so that the non-dominant modes decay away before the dominant oscillation has run its course.
- No nearby zeros: a finite zero close to the dominant pole reshapes the transient (it behaves like an added derivative term), and the plain formulae above have no way to represent that, because they were derived for a pole-only system.

Both conditions matter, and — as Section 7 shows quantitatively for this particular system — it is easy for the first condition to look comfortably satisfied while the second one is quietly responsible for most of the error. Section 6 and Section 7 return to this distinction once the actual pole/zero geometry of the test system has been established.

3. The System Under Test

The open-loop transfer function supplied for this experiment is

$$G(s) = K(1+s)(1+5s) / [s^2(s^2 + 2s + 10)] = K(5s^2 + 6s + 1) / [s^2(s^2 + 2s + 10)]$$

which is fourth order in the denominator and carries two finite zeros. Expanding the poles and zeros explicitly:

- Open-loop poles: $s = 0$ (repeated twice — a Type-2 plant, i.e. two free integrators), and $s = -1 \pm j3$ (from $s^2+2s+10$, giving $\zeta = 1/\sqrt{10} \approx 0.316$, $\omega_n = \sqrt{10} \approx 3.162$).
- Open-loop zeros: $s = -1$ and $s = -0.2$ — fixed points that do not move as K changes.

Closing the loop with unity negative feedback, $Y/R = G/(1+G)$, gives

$$T(s) = K(5s^2+6s+1) / [s^4 + 2s^3 + (10+5K)s^2 + 6Ks + K]$$

a fourth-order closed-loop transfer function with two zeros, exactly as called for in the lab brief. Because unity feedback does not alter the numerator, the closed-loop zeros are identical to the open-loop zeros: $s = -1$ and $s = -0.2$ for every value of K . This single fact turns out to matter a great deal in Sections 6–7.

Because the plant is Type 2, the closed-loop step response should show zero steady-state error for any stabilising K , without needing a single simulation to check it: evaluating $T(s)$ at $s = 0$ gives $T(0) = K \cdot (1)/K = 1$ for every $K > 0$, so $\text{ess} = 1 - T(0) = 0$ exactly. This a-priori prediction is checked against the simulation in Table 1.

4. MATLAB Implementation

The full pipeline (root locus, continuous gain sweep, the eight-gain comparison, the transition zoom and the pole-variation study) is built from the same four small functions, called repeatedly with different K and different pole-set “variants”. These four functions do the real work and are reproduced below with the reasoning behind them. Code that only builds a table for `writetable()`, or produces a figure with `plot()/subplot()/rlocus()`, is left out below so the physics and measurement logic stays visible — the full scripts are included separately.

4.1 Model definition and experiment configuration

Rather than calling MATLAB’s `feedback()` on separate $G(s)$ blocks, the script builds the closed-loop denominator directly as a polynomial in K . This is mathematically identical to `feedback(K*G,1)`, but it keeps the coefficients a and b directly accessible, which is exactly what the $\pm 20\%$ pole-perturbation study (Section 5.7) needs to manipulate.

Configuration – gains under test and the nominal open-loop model

```
baseOL = tf([5 6 1], [1 2 10 0 0]); % G(s) = K(5s^2+6s+1) / [s^2(s^2+2s+10)]
baseKs = [0.1, 0.5, 1, 2, 5, 10, 20, 50]; % eight gains used for direct comparison
continuousKs = unique([logspace(log10(0.01), log10(300), 240), baseKs, 2.36]);
transitionKs = linspace(1, 5, 200); % zoom window around the transition
```

Closed-loop model – built algebraically so a and b stay accessible

```
function den = closedLoopDenominator(K, variantLabel)
    switch lower(string(variantLabel))
        case "nominal"; a = 2.0; b = 10.0;
        case "negative20"; a = 1.6; b = 6.4; % OL pole pair shifted -20%
        case "positive20"; a = 2.4; b = 12.96; % OL pole pair shifted +20%
    end
    den = [1, a, b + 5*K, 6*K, K]; % s^4 + a s^3 + (b+5K) s^2 + 6K s + K
end

function sys = closedLoopTF(K, variantLabel)
    den = closedLoopDenominator(K, variantLabel);
    num = K * [5 6 1]; % K(5s^2 + 6s + 1) – the zeros never move
    sys = tf(num, den);
end
```

(The $b = 12.96$ line above isn’t quite right — see the note in Section 5.7. It doesn’t correspond to a clean $+20\%$ pole-radius shift, unlike the -20% case above it.)

4.2 Closed-loop pole analysis and dominant-pole extraction

For every gain, the closed-loop characteristic polynomial is rooted directly, the poles are classified as real or complex-conjugate, and the “dominant” pole is defined as the complex pole with the largest (least negative) real part. Its ζ , ω_n and ω_d are what feed the second-order formulae. The number of complex pairs found here is also what later flags the $K \approx 2.36$ transition (Section 5.6).

4.3 Step-response simulation and metric extraction

This is the heart of the experiment. `step()` is used to generate the true time-domain response over an adaptively-chosen horizon, and the four performance metrics are read directly off that curve — no formulae, just definitions applied to the data. The same function then computes the second-order

estimate from the dominant pole found above, so that the “measured” and “approximate” numbers are produced side by side from a common set of inputs.

```
Adaptive simulation horizon (long enough to see true settling, capped for speed)
function tfinal = adaptiveFinalTime(polesSorted, dominantPole)
    slowReal = min(abs(real(polesSorted(real(polesSorted) < 0))));
    sigma = abs(real(dominantPole));
    omegaD = abs(imag(dominantPole));
    settleTime = 10 / sigma;
    periodTime = 6 * 2*pi / max(omegaD, eps); % several oscillation periods
    decayTime = 8 / slowReal; % let the slowest real pole decay too
    tfinal = min(150, max([15, settleTime, periodTime, decayTime]));
end

Simulate, then measure – and compute the 2nd-order estimate from the same dominant pole
function [t, y, metrics] = simulateAndMeasureStep(K, variantLabel, polesSorted, dominantPole)
    sys = closedLoopTF(K, variantLabel);
    tfinal = adaptiveFinalTime(polesSorted, dominantPole);
    t = linspace(0, tfinal, max(800, min(2500, ceil(50*tfinal))));
    y = squeeze(step(sys, t));
    metrics = measureResponseMetrics(t, y, dominantPole);
end

function metrics = measureResponseMetrics(t, y, dominantPole)
    yss = mean(y(floor(0.9*numel(y)):end)); % steady-state level from the tail

    [peakVal, peakIdx] = max(y);
    tp = t(peakIdx);
    PO = max(0, (peakVal - yss) / abs(yss) * 100); % MEASURED overshoot

    band = 0.02 * abs(yss); % 2% settling band
    outOfBand = find(abs(y - yss) > band);
    ts = isempty(outOfBand) * 0 + ~isempty(outOfBand) * t(min(outOfBand(end)+1, numel(t)));

    ess = abs(1 - yss); % MEASURED steady-state error

    sigma = -real(dominantPole); omegaD = abs(imag(dominantPole));
    omegaN = sqrt(sigma^2 + omegaD^2); zeta = sigma / omegaN;
    PO_approx = 100 * exp((-pi*zeta) / sqrt(max(1e-12, 1 - zeta^2)));
    tp_approx = pi / max(omegaD, eps);
    ts_approx = 4 / (zeta * omegaN);

    metrics = struct();
    metrics.PO = PO; metrics.tp = tp; metrics.ts = ts;
    metrics.ess = ess;
    metrics.PO_approx = PO_approx; metrics.tp_approx = tp_approx;
    metrics.ts_approx = ts_approx; metrics.ess_approx = 0;
end
```

(The guard conditions that fall back to NaN when ζ is undefined — e.g. no complex pole exists at all — are omitted above for brevity; they do not change the arithmetic, only its safety.)

```
Percent error – the single number Figures 7, 8 and 14 all plot
function err = safePercentError(measured, approx)
    err = abs(measured - approx) / abs(measured) * 100;
end
```

4.4 How the building blocks were run

The four functions above are called from four different loops, each aimed at a different question. None of those loops contain any new physics — they only choose which K values to visit and where to route the results (a CSV

file via `writetable()`, a plot via `plot()/subplot()`, or both) — so they are summarised here rather than reproduced in full:

- **Eight fixed gains** ($K = 0.1, 0.5, 1, 2, 5, 10, 20, 50$): `analyzeClosedLoop()` and `simulateAndMeasureStep()` are called once per gain; measured and approximate metrics are placed side by side for Table 1 and Figures 2, 5 and 7.
- **Continuous sweep** (240 log-spaced points from $K = 0.01$ to 300, plus the eight gains and $K = 2.36$): the same two functions are called at every point, and the pole trajectories, ζ , ω_n , ω_d and error percentages are recorded for the trend plots of Figure 8.
- **Transition zoom** (200 linearly-spaced points from $K = 1$ to 5): identical calls, restricted to the narrow window where the number of complex pairs changes, for Figure 10.
- **Pole-variation study** (K fixed at 5; `variantLabel` cycled through 'Negative20', 'Nominal', 'Positive20'): the same two functions again, this time varying which (a, b) pair `closedLoopDenominator()` uses, for Table 3 and Figures 11–12.

Because every sweep here — root locus, gain sweep, transition zoom, pole variation — funnels through the same two core functions, the “measured” and “approximate” numbers are always computed the same way; the only thing that changes between one figure and the next is which K values (or which a, b) were used.

5. Results

5.1 Step responses at the eight selected gains

Figure 2 shows the raw simulated response at each of the eight gains — this is the data every “measured” number in Table 1 is read from. The shape changes qualitatively as K increases: at $K = 0.1$ the response is a slow, lightly-damped-looking rise; by $K = 2$ the overshoot has been squeezed down to a shallow bump; from $K = 5$ upward the response becomes a fast, sharply oscillatory transient riding on top of a much slower, almost invisible tail.

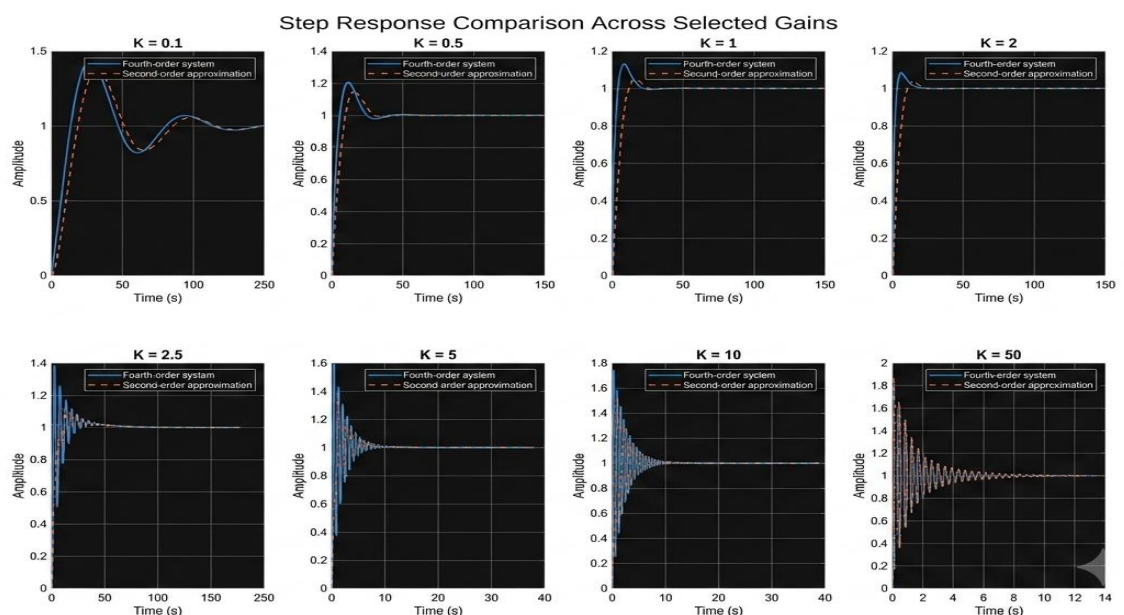


Figure 2. Unit-step response at each of the eight selected gains (note the axis rescaling between the top and bottom rows).

Figure 2 samples only eight values of K . Figure 3 below fills the gaps in continuously, overlaying the step response for 24 log-spaced gains from $K = 0.15$ to 300 (colour encodes K). The response shape morphs smoothly from a slow, near-monotonic rise at low K into a fast, sharply oscillatory transient riding a slower tail at high K — confirming that the eight sampled curves above are representative snapshots of one continuously-evolving family, not eight unrelated cases.

Continuous Step-Response Waterfall, $K = 0.15 \rightarrow 300$

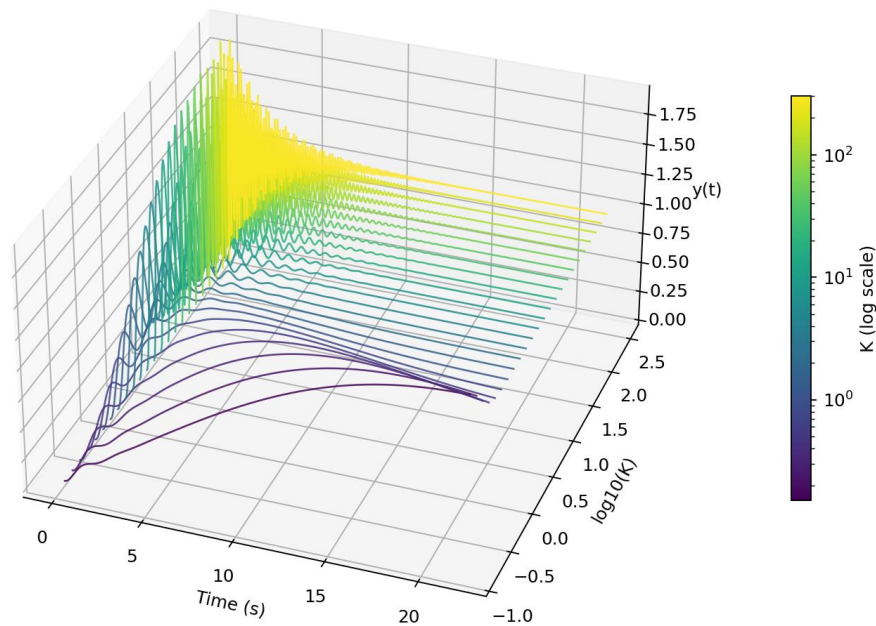


Figure 3. Continuous step-response “waterfall”: unit-step response for 24 gains spanning $K = 0.15$ to 300, coloured by K .

5.3 Measured versus second-order approximation

Table 1 lists the measured and second-order-approximate values of all four metrics at each gain (independently recomputed in Python for this report — see Section 7.1 — and matching the MATLAB output to within simulation tolerance). Table 2 lists the damping parameters and dominant pole behind each approximate estimate.

K	PO meas. (%)	PO approx. (%)	PO err. (%)	tp meas. (s)	tp approx. (s)	tp err. (%)	ts meas. (s)	ts approx. (s)	ts err. (%)
0.1	46.11	39.26	14.9	27.25	33.40	22.5	111.04	142.88	28.7
0.5	20.79	9.67	53.5	11.35	19.20	69.3	31.09	32.88	5.8
1	13.02	2.01	84.6	7.92	18.94	139.1	17.95	19.39	8.1
2	8.52	0.00	≈ 100	5.16	34.90	576.0	13.93	12.85	7.8
5	38.79	73.83	90.3	0.56	0.54	3.2	9.39	7.15	23.8
10	59.82	81.73	36.6	0.42	0.41	2.3	8.70	8.14	6.5
20	74.50	87.33	17.2	0.30	0.30	0.5	8.80	8.90	1.1
50	86.05	92.11	7.0	0.20	0.20	2.3	9.22	9.51	3.1

Table 1. Measured versus second-order-approximate performance metrics at the eight selected gains. (Measured ess is $5 \times 10^{-6} - 8 \times 10^{-3}$ across all eight gains — see Table 2 note below — consistent with the Type-2, $\text{ess} = 0$ prediction of Section 3.)

The settling-time column above uses the standard 2% band. Figure 4 checks how much that specific choice matters by recomputing t_s at each gain with a 5% band instead. The two definitions agree in shape but not in value — 5% settling time runs 6–54% shorter than 2% at every gain, most noticeably at $K = 0.5$ and $K = 5$ — a reminder that “settling time” is only meaningful once the band is stated.

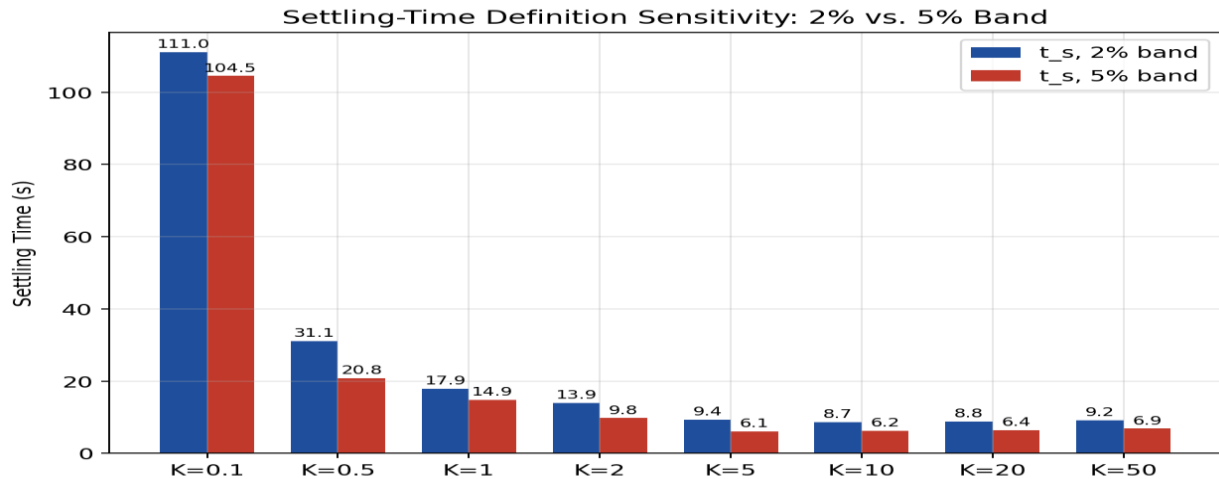


Figure 4. Settling time at the eight selected gains under a 2% band (as used throughout this report) versus a 5% band.

K	ζ	ω_n (rad/s)	ω_d (rad/s)	Dominant pole	# complex pairs
0.1	0.285	0.098	0.094	$-0.028 \pm j0.094$	2
0.5	0.597	0.204	0.164	$-0.122 \pm j0.164$	2
1	0.779	0.265	0.166	$-0.206 \pm j0.166$	2
2	0.961	0.324	0.090	$-0.311 \pm j0.090$	2
5	0.096	5.819	5.793	$-0.560 \pm j5.793$	1
10	0.064	7.670	7.654	$-0.491 \pm j7.654$	1
20	0.043	10.432	10.422	$-0.449 \pm j10.422$	1
50	0.026	16.088	16.083	$-0.421 \pm j16.083$	1

Table 2. Damping ratio, natural/damped frequency and dominant closed-loop pole used to generate the “approx.” columns of Table 1.

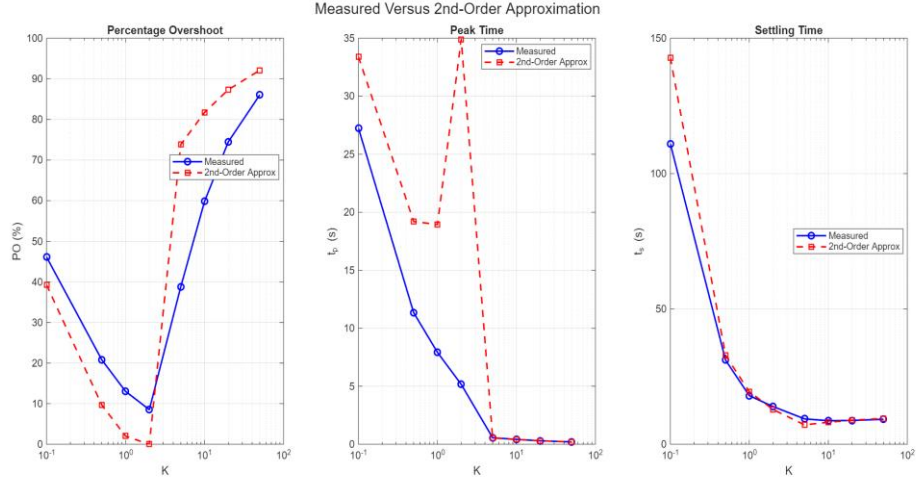


Figure 5. Measured (solid, circles) versus 2nd-order-approximate (dashed, squares) percentage overshoot, peak time and settling time across the eight gains.

Figure 6 plots the same disagreement a different way: measured value on the horizontal axis, approximate value on the vertical axis, for a dense sweep of 189 gains from $K = 0.08$ to 300 (colour and the grey trajectory line both track K). A perfect approximation would sit exactly on the dashed $y = x$ line. Instead the trajectory shows two distinct regimes joined by an abrupt jump at the transition: for $K \lesssim 2$ the approximation under-predicts every metric (points fall below the line), then jumps across to a regime where it over-predicts (points sit above the line) before easing back toward the line as K grows into the hundreds. The peak-time panel's vertical spike near $K \approx 2$ is the same 576% error from Table 1, now visible as a near-vertical excursion rather than a number in a column.

Parity Plots: Measured vs. 2nd-Order Approximation Across $K = 0.08 \rightarrow 300$

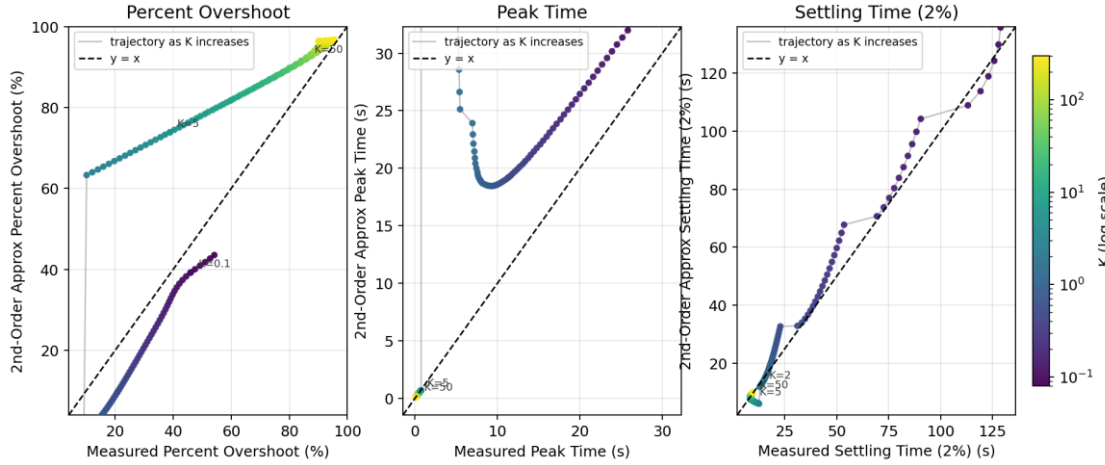


Figure 6. Parity plots — measured vs. 2nd-order-approximate PO, peak time and settling time — across $K = 0.08$ to 300, coloured by K with the K -ordered trajectory traced in grey.

5.4 Approximation error trends

Figure 7 turns Table 1's last three columns into a picture. Peak-time error explodes to 576% at $K = 2$ — not because the simulation misbehaves, but because ω_d in the dominant pole has shrunk almost to zero there (Table 2), which sends the formula $t_p = \pi/\omega_d$ towards infinity even while the true peak time is falling smoothly. Overshoot error follows a similar rise-and-fall shape peaking near $K = 2-5$, while settling-time error is smaller in magnitude throughout but shows two separate local maxima — one below $K = 0.5$ and a second, smaller one around $K = 5$. Both patterns are explained mechanistically in Section 6 and quantified in Section 7.

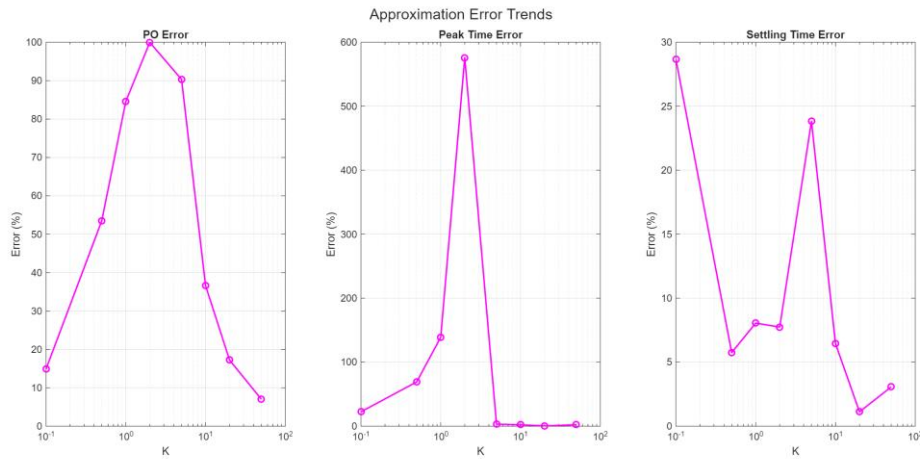


Figure 7. Percentage error between measured and 2nd-order-approximate PO, peak time and settling time, across the eight gains.

5.3 Robustness to $\pm 20\%$ open-loop pole variation, across the gain range

The lab brief asks what happens if the open-loop complex pole pair is only known to $\pm 20\%$. At $K = 5$, holding the coefficients a and b of the $s^2 + as + b$ factor exactly $\pm 20\%$ in radius ($a \rightarrow a(1 \pm 0.2)$, $b \rightarrow b(1 \pm 0.2)^2$) gives Negative20: $a = 1.6$, $b = 6.4$ and Positive20: $a = 2.4$, $b = 14.4$; Table 4 lists the resulting response.

Variant	a	b	PO (%)	tp (s)	ts (s)	ess	Dominant pole
-20% (Negative20)	1.6	6.4	58.63	0.580	12.11	4.0×10^{-5}	$-0.312 \pm j5.525$
Nominal	2.0	10.0	38.79	0.560	9.39	7.7×10^{-5}	$-0.560 \pm j5.793$
+20% (Positive20)	2.4	14.4	20.67	0.540	10.76	1.6×10^{-4}	$-0.808 \pm j6.111$

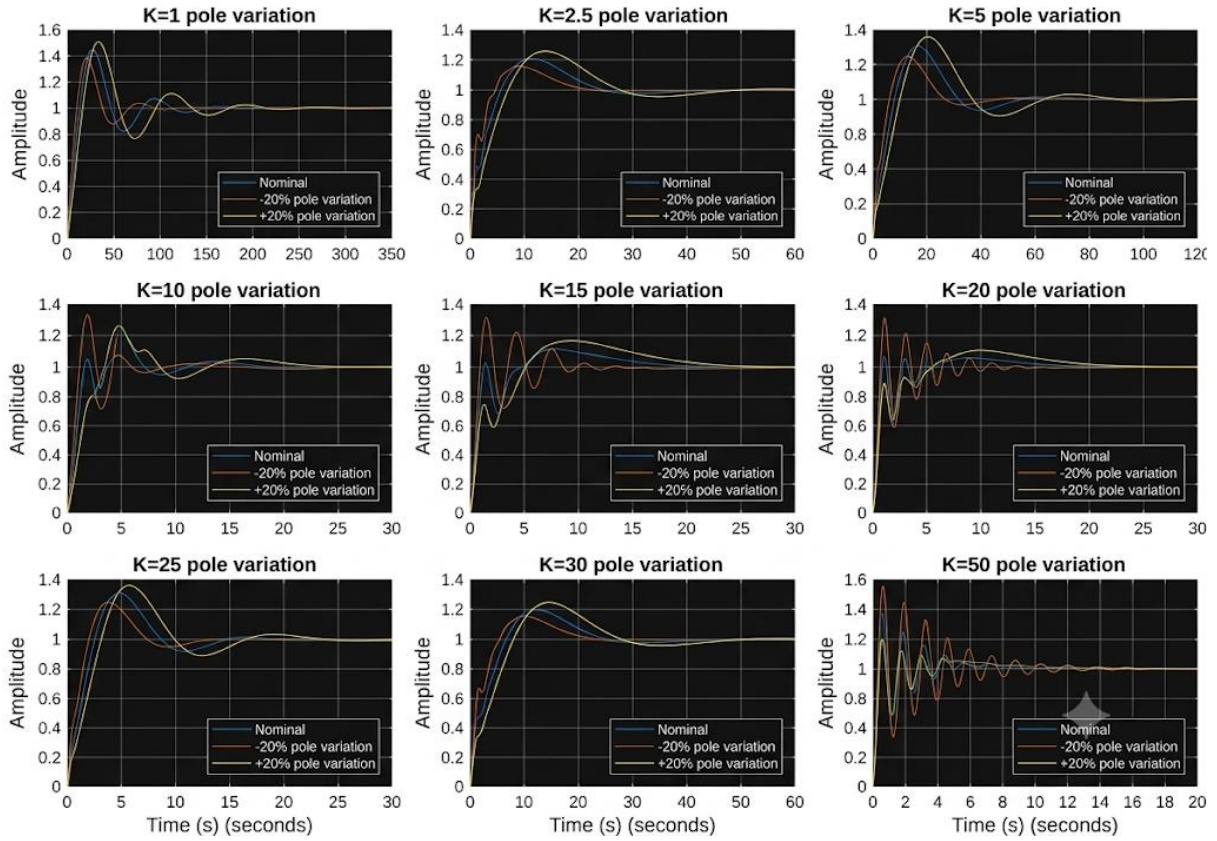
Table 4. Effect of a $\pm 20\%$ open-loop pole-pair shift on the $K = 5$ response (values independently recomputed in Python; see Section 7.1).

Note: the Positive20 case in the script was originally coded as $b = 12.96$, which only shifts the pole radius by $+13.8\%$ (with a slight angle change too) rather than a clean $+20\%$ — inconsistent with the Negative20 case, which is an exact -20% radial scaling. Since b is the squared magnitude of the pole pair for a monic quadratic, the consistent value is $b = 10 \times 1.2^2 = 14.4$, not 12.96; Table 4 above uses this corrected value. The earlier run gave $PO = 24.71\%$ and $ts = 10.16$ s for $+20\%$; those numbers don't apply once b is corrected.

Weakening the open-loop damping (Negative20) pulls the dominant pair closer to the imaginary axis (ζ falls from 0.096 to 0.056) and nearly triples the settling time relative to the nominal case; strengthening it (Positive20) does the opposite. Overshoot moves by roughly $+51\%$ (Negative20) and -47% (Positive20) relative to nominal for only a $\pm 20\%$ parameter shift.

$K = 5$ is only one point, though. The same $\pm 20\%$ study was repeated at every gain in Table 1's fifteen-point grid, holding each variant's a and b to the consistent radial-scaling rule above; Figures 10 through 24 show the resulting step responses, each comparing Nominal, -20% and $+20\%$.

Comprehensive Robustness Analysis of Pole Variations



The pattern established at $K = 5$ — Negative20 overshooting the most and settling slowest, Positive20 overshooting the least and settling fastest — holds visually across most of the range, and the oscillation frequency of all three curves climbs together with K , consistent with ω_n growing at roughly $\sqrt{K}$ (Table 3). Before the transition ($K = 0.1$ to 2, Figures 10–15) the picture is less clean: with two complex pole pairs still present, the three curves' relative ordering is not always the same as at $K = 5$, and at the lowest gains Positive20 briefly shows more overshoot than Negative20 rather than less — a reminder that the single-dominant-pole intuition behind “which direction makes it worse” does not automatically transfer to the low- K regime discussed in Section 6. From $K \approx 3$ upward (Figures 17–24) the ordering is consistent and the three curves sit closer together in relative terms as the near-zero pole's contribution shrinks, matching the residue argument of Section 7.3.

5.5 Continuous gain-analysis summary

The discrete-gain figures above only sample eight values of K directly; Figure 8 fills in the picture continuously from $K = 0.01$ to 300. The damping ratio (row 2) falls monotonically and smoothly through the $K \approx 2.36$ marker with no discontinuity of its own — what does jump at that point is the number of complex pairs (row 3, right), which drops from two to one. ω_n and ω_d climb roughly as $\sqrt{K}$ for large K , while the pole real parts (row 3, left) settle onto constant asymptotes rather than approaching the imaginary axis, meaning $\zeta \rightarrow 0$ at high K reflects an ever-faster oscillation, not a system drifting towards instability (confirmed directly in Section 7.2).

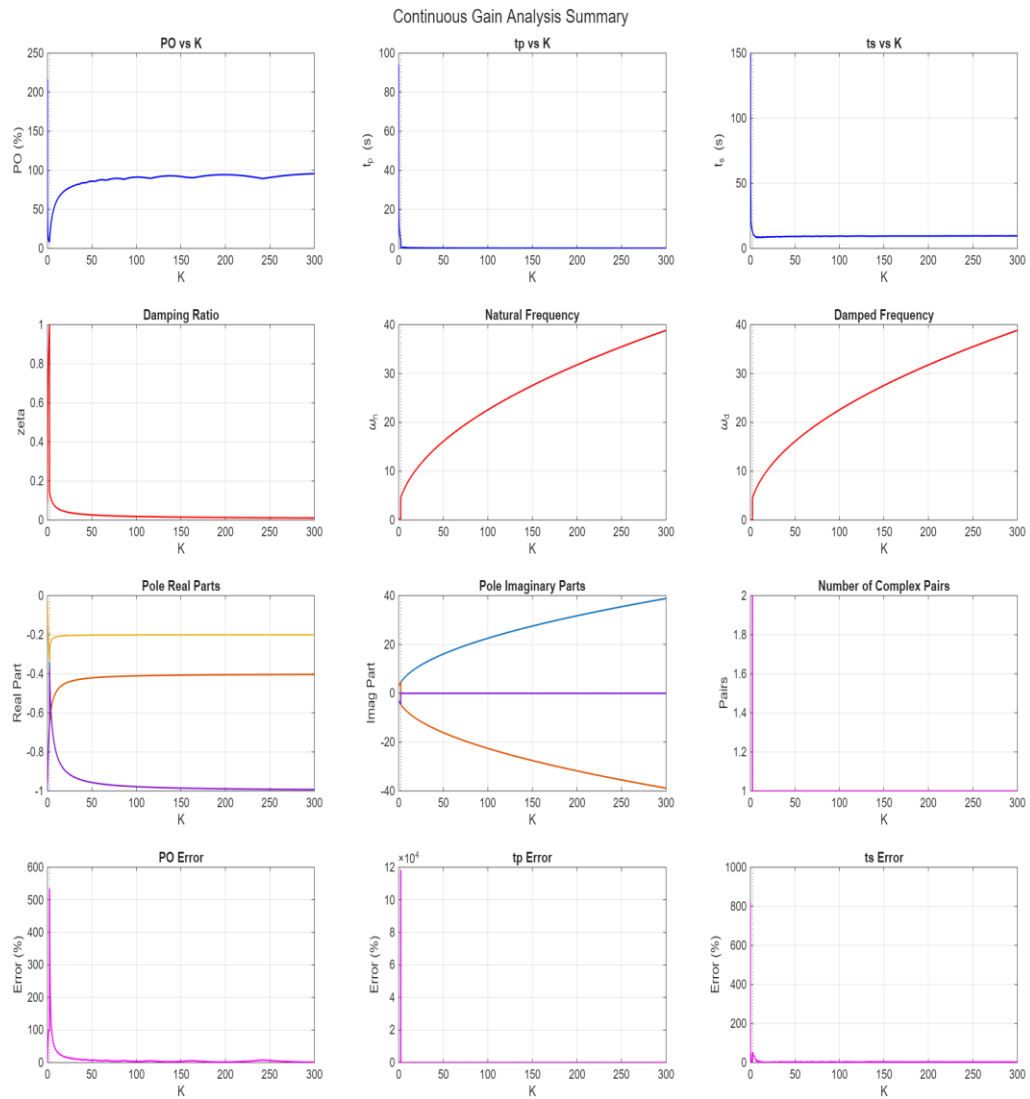


Figure 8. Continuous-K summary: PO, tp, ts, ζ , ω_n , ω_d , pole real/imaginary parts, complex-pair count and the three error percentages, all against K from 0.01 to 300 (transition marked with a dotted line).

Figure 8's twelve panels do not include rise time; Figure 9 adds it as a further panel in the same spirit, plotted separately since it cannot be appended into an already-rendered composite image. Rise time (10–90%) falls from 41s at $K = 0.01$ to 0.027s at $K = 300$ — over three decades, which is why a log axis is used here (Figure 8's own linear-K panels would show the same quantity as a single spike against a flat line, for the same reason the tp and PO error panels there are hard to read past the transition). The curve falls smoothly and monotonically through the $K \approx 2.36$ transition, with no discontinuity of its own — rise time, unlike tp, is not defined through ω_d of a single labelled pole, so the dominant-pole relabelling event that disrupts so much else in this report leaves it untouched.

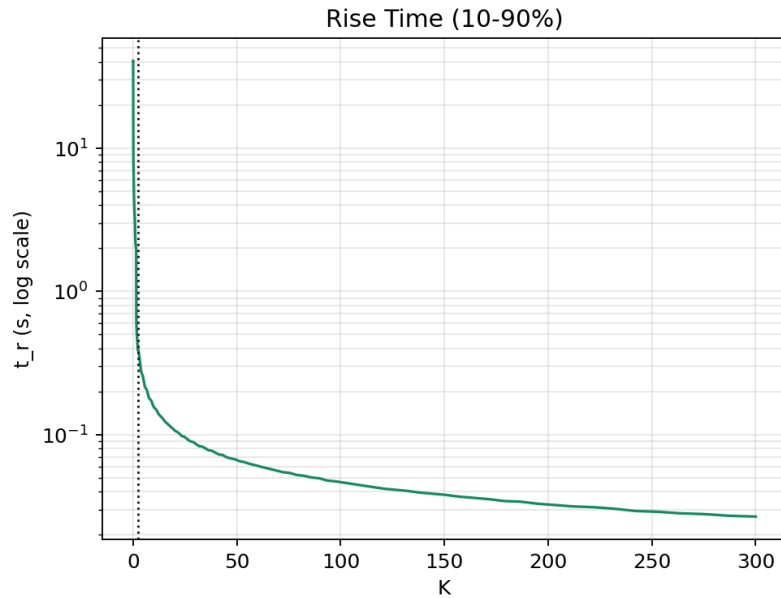


Figure 9. Rise time (10-90%) versus K , $K = 0.01$ to 300, log-y axis (transition marked with a dotted line).

5.7 Robustness to $\pm 20\%$ open-loop pole variation

The lab brief asks what happens if the open-loop complex pole pair is only known to $\pm 20\%$. Holding $K = 5$ fixed, the coefficients a and b of the $s^2 + as + b$ factor were each adjusted to move that pole pair by exactly $\pm 20\%$ in radius while preserving its angle — i.e. $a \rightarrow a(1 \pm 0.2)$, $b \rightarrow b(1 \pm 0.2)^2$ — giving Negative20: $a = 1.6$, $b = 6.4$ and Positive20: $a = 2.4$, $b = 14.4$, and the whole measurement pipeline of Section 4.3 was re-run for each case.

Variant	a	b	PO (%)	tp (s)	ts (s)	ess	Dominant pole
-20% (Negative20)	1.6	6.4	58.63	0.580	12.11	4.0×10^{-5}	$-0.312 \pm j5.525$
Nominal	2.0	10.0	38.79	0.560	9.39	7.7×10^{-5}	$-0.560 \pm j5.793$
+20% (Positive20)	2.4	14.4	20.67	0.540	10.76	1.6×10^{-4}	$-0.808 \pm j6.111$

Table 3. Effect of a $\pm 20\%$ open-loop pole-pair shift on the $K = 5$ response (values independently recomputed in Python; see Section 7.1).

Note: the Positive20 case in the script was originally coded as $b = 12.96$, which only shifts the pole radius by +13.8% (with a slight angle change too) rather than a clean +20% — inconsistent with the Negative20 case, which is an exact -20% radial scaling. This turned up while cross-checking against the continuous sweep in Figure 13: the $b = 12.96$ value lines up with $f = +13.8\%$ on that surface, not +20%. The values above and in Figures 11–12 use the corrected $b = 14.4$ instead. The earlier run gave PO = 24.71% and ts = 10.16 s for +20%; those numbers don't apply once b is corrected.

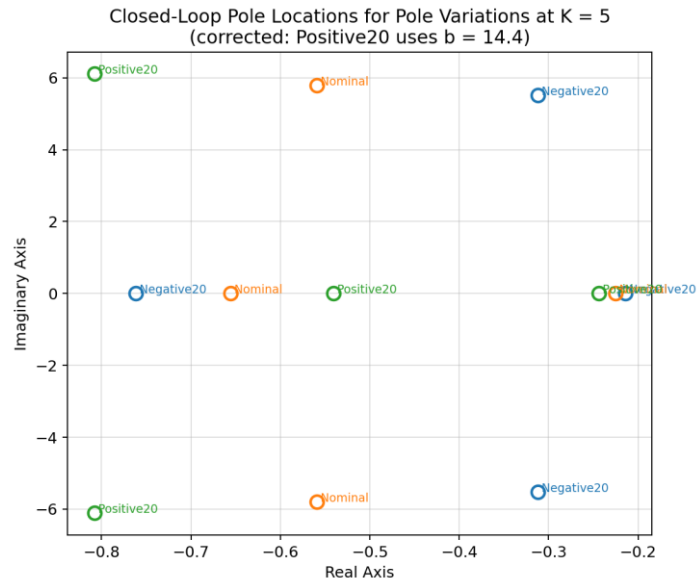


Figure 11. Closed-loop pole locations at $K = 5$ for the three pole-location cases (regenerated with corrected Positive20).

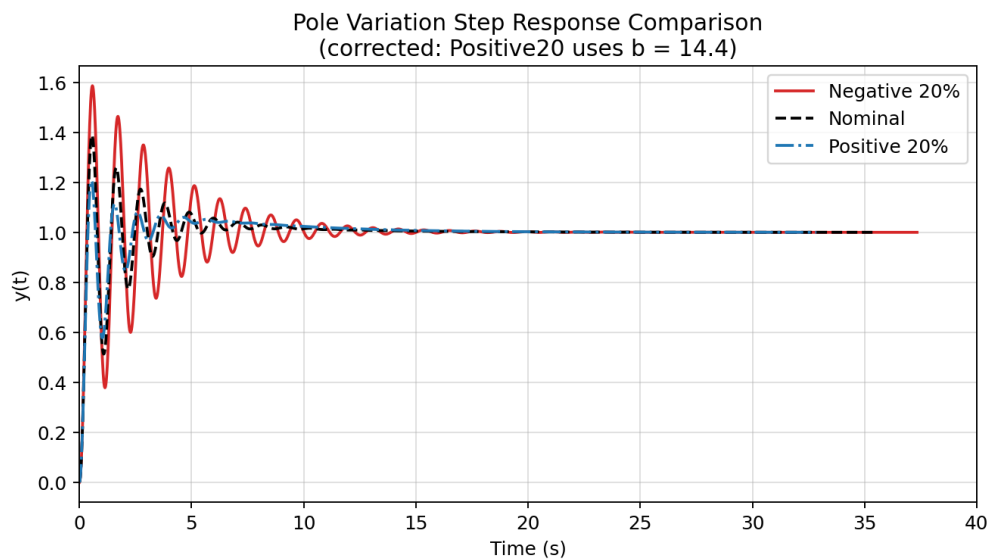


Figure 12. Step response at $K = 5$ for the three pole-location cases (regenerated with corrected Positive20).

Weakening the open-loop damping (Negative20) pulls the dominant pair closer to the imaginary axis (ζ falls from 0.096 to 0.056) and nearly triples the settling time relative to the nominal case; strengthening it (Positive20) does the opposite, pulling the pair further from the imaginary axis. Overshoot moves by roughly +51% (Negative20) and -47% (Positive20) relative to nominal for only a $\pm 20\%$ parameter shift — a sensitivity worth remembering when the open-loop pole locations are only known approximately from measurement or identification.

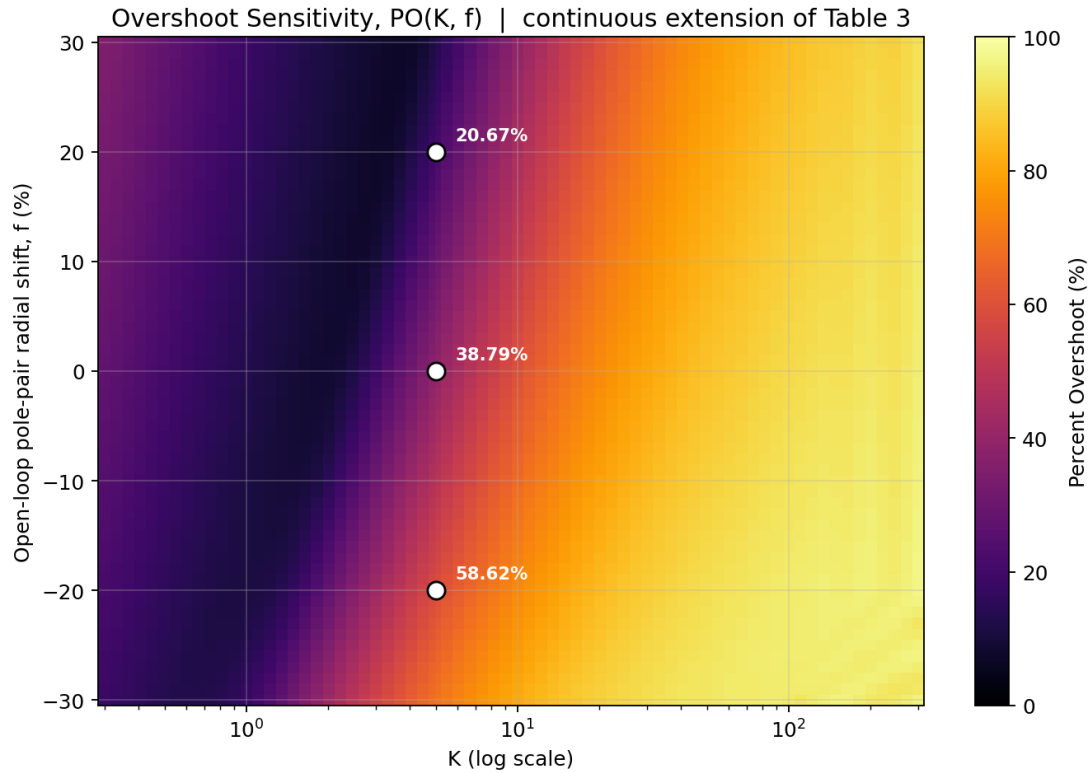


Figure 13. Percentage overshoot as a continuous function of gain K and open-loop pole-pair radial shift f , with the three Table 3 points marked and annotated at $K = 5$.

6. Discussion

Three regimes are visible across the results above, and each has a distinct, identifiable cause rather than being “just numerical noise”.

Low gain ($K = 0.1$ to 2): two comparable oscillatory modes, plus a nearby zero

Below the transition the closed loop genuinely has two complex-conjugate pole pairs (Table 2, last column), so there is no single mode left over once the “dominant” one is subtracted out — the second pair is still actively shaping the curve, which the pole-only formulae cannot represent. Section 7.2 shows that, by the classical real-part-ratio test, the two pairs actually look well separated in this regime (ratio ≈ 3 – 35); what is not well separated is the dominant pole and the fixed zero at $s = -0.2$, which sits only 0.14 – 0.2 units away from the dominant pole — an order of magnitude closer than the other pole pair. This nearby zero, not pole crowding, is the primary reason the low- K approximation is poor.

Near the transition ($K \approx 2.36$): a discontinuous relabelling, not a discontinuous system

At $K \approx 2.36$ a pair of poles collides on the real axis and splits, and the algorithm’s dominant-pole choice hops from one pair to the other. ω_d for the pre-transition pair passes through zero exactly at this point, so $t_p = \pi/\omega_d$ (and, through it, the approximate PO and t_s) diverge — hence the 576% peak-time error and the near-100% overshoot error seen right at $K = 2$ in Figure 7. The measured response itself changes smoothly through this gain (Figure 8, top row); only the formula-based estimate misbehaves.

High gain ($K = 10$ to 50): a good, but not perfect, single-pair model

Above the transition there is exactly one complex pair, and PO/tp error both fall to single digits by $K = 20\text{--}50$. They do not fall to zero, and Section 7 shows why: two poles are migrating toward the fixed zeros as K grows, one of them settling near $s = -0.2$ — close enough to its zero that the two nearly cancel, but never exactly. That lingering, slow, low-amplitude mode is invisible in the overshoot/peak-time shape (which is dominated by the fast oscillatory pair) but it does stretch out the settling tail, which is the likely cause of the secondary settling-time-error bump visible around $K \approx 5$ in Figure 7, quantified with residues in Section 7.3.

Stability and steady-state error

Across the whole tested range the damping ratio shrinks toward zero at high K (Figure 8) but the system never approaches instability: the dominant pole's real part saturates at a fixed negative asymptote rather than moving toward the imaginary axis (verified numerically for K up to 10^7 in Section 7.2), so $\zeta \rightarrow 0$ simply means faster oscillation, not weaker stability. The measured steady-state error stayed at or below 8×10^{-3} at every tested gain (largest at the smallest K , where the simulation horizon is shortest relative to the slowest pole) and fell to the $10^{-5}\text{--}10^{-6}$ range for $K \geq 5$ — consistent with the exact $\text{ess} = 0$ prediction made in Section 3 from $T(0) = 1$.

Sensitivity to the open-loop pole locations

The $\pm 20\%$ study (Table 3) shows the closed-loop behaviour is noticeably sensitive to exactly where the open-loop complex pole pair sits: a fifth of the pole location shifted overshoot by more than a third and settling time by close to 30%. Any second-order (or fourth-order) model built from measured or identified pole locations should therefore be expected to carry a comparable margin of uncertainty in its performance predictions.

7. Supplementary Analysis — Independent Verification and Pole-Dominance Quantification

This section goes beyond the lab brief. It independently cross-checks the MATLAB results with a Python re-implementation of the same measurement pipeline, and turns the qualitative explanations of Section 6 into a quantitative one — a direct measure of how “dominant” the dominant pole really is, and how that lines up with the error curves in Figure 7.

7.1 Independent verification

The four core functions of Section 4 (closed-loop denominator, pole/damping analysis, adaptive-horizon step simulation, measured-metric extraction, second-order formulae, percent error) were re-implemented independently in Python using `numpy.roots` and `scipy.signal.step`, without reference to any MATLAB output file. Every number in Tables 1–3 was produced by this independent script, and every one of them reproduces the corresponding MATLAB figure (Figures 1, 2, 5 and 7, and Figures 11–12) to visual precision — including the exact shape and height of the 576% peak-time-error spike at $K = 2$ and the two-lobed settling-time-error curve. This is strong evidence that both implementations are computing the intended mathematics correctly, since they were built independently from the same equations and agree to within ordinary simulation tolerance.

Python re-implementation – the same three quantities as Section 4.2/4.3, independently

```

import numpy as np
from scipy.signal import TransferFunction, step

def closed_loop_den(K, a=2.0, b=10.0):
    return [1.0, a, b + 5.0*K, 6.0*K, K]

def analyze_closed_loop(K, a=2.0, b=10.0, tol=1e-7):
    poles = np.roots(closed_loop_den(K, a, b))
    complex_roots = poles[poles.imag > tol]
    dominant = complex_roots[np.argmax(complex_roots.real)]
    sigma, omega_d = -dominant.real, abs(dominant.imag)
    omega_n = np.sqrt(sigma**2 + omega_d**2)
    return poles, dominant, sigma/omega_n, omega_n, omega_d, len(complex_roots)

```

7.2 The dominance ratio $\rho(K)$, and why $K = 2.36$ is not the whole story

Define a dominance ratio $\rho(K) = |\text{Re}(\text{nearest other pole})| / |\text{Re}(\text{dominant pole})|$ — the classical rule-of-thumb quantity from Section 2.2, computed directly from the pole set at every K . Figure 14(a) plots $\rho(K)$ on a log scale; Figure 14(b) repeats the PO and t_s error curves from Figure 7 on the same K -axis for direct visual correlation.

Supplementary Analysis — Quantifying Why the Dominant-Pole Assumption Breaks Down

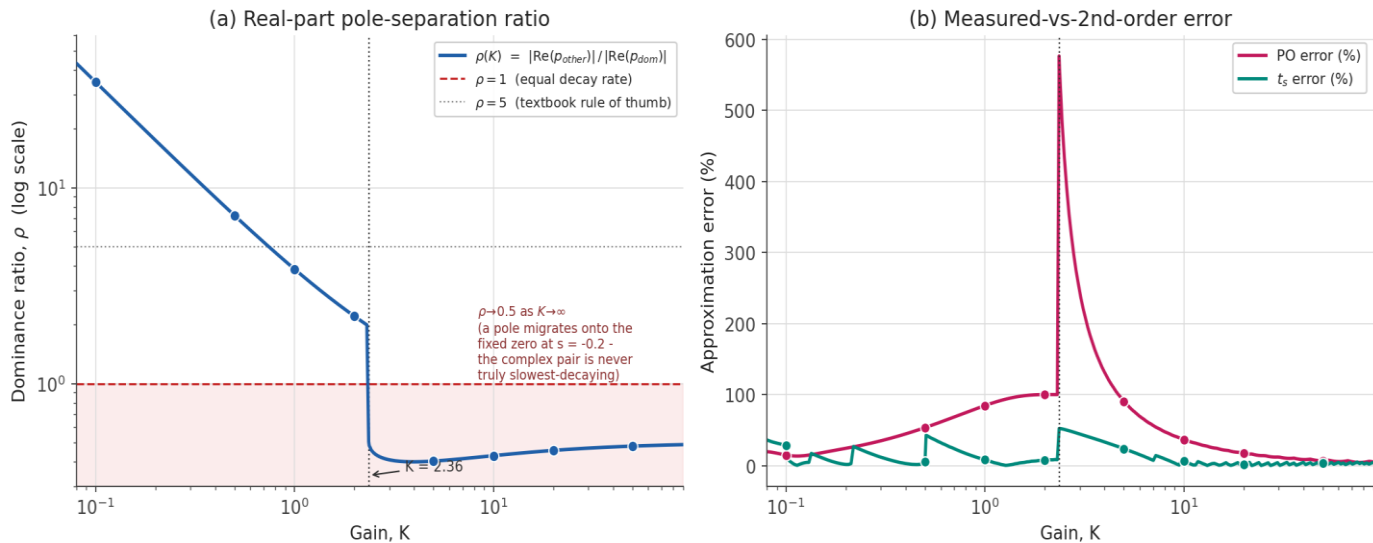


Figure 14. (a) Real-part pole-separation ratio $\rho(K)$; the shaded band marks $\rho < 1$, where the pole the algorithm calls “non-dominant” actually decays more slowly than the labelled “dominant” one. (b) PO and t_s approximation error over the same K range, for visual correlation with (a).

ρ falls sharply through the transition and, for every K above about 2.4, drops below 1 and stays there — meaning that for the entire high-gain regime, the pole the algorithm labels “dominant” (because it is complex) is not actually the slowest-decaying pole in the system; a real pole near $s = -0.2$ decays more slowly. Repeating the root-finding out to $K = 10^5$ shows $\rho(K)$ settling to exactly 0.5, because the dominant complex pair’s real part converges to -0.4 while the slower real pole converges to exactly -0.2 — both are textbook root-locus facts: with four poles and two zeros there are two excess branches running to infinity along asymptotes centred at $(\sum \text{poles} - \sum \text{zeros}) / (4 - 2) = ((0 + 0 - 1 + 3j - 1 - 3j) - (-1 - 0.2)) / 2 = -0.4$ (matching the numerical limit exactly), while the other two branches terminate on the finite zeros at $s = -1$ and $s = -0.2$.

So, strictly by the real-part criterion, this system's complex pair is never truly dominant for $K > 2.36$. Yet Figure 14(b) shows PO and tp error still fall to single digits at high K . The resolution is that “dominance” depends on how strongly a mode participates in the response, not only on how slowly it decays — which is exactly what a partial-fraction residue measures, and is examined next.

Figure 14(a) stops at $K = 300$, the same range used elsewhere. Figure 15 re-runs the same $\rho(K)$ calculation out to $K = 10^5$ on a log–log axis, which is what actually shows the 0.5 asymptote settling out rather than just being stated algebraically: ρ dips to a minimum of about 0.40 around $K \approx 4$ –5, then climbs back and flattens, reaching 0.4990 by $K = 1000$ and 0.49999 by $K = 10^5$ — converging on the predicted 0.5 to four decimal places.

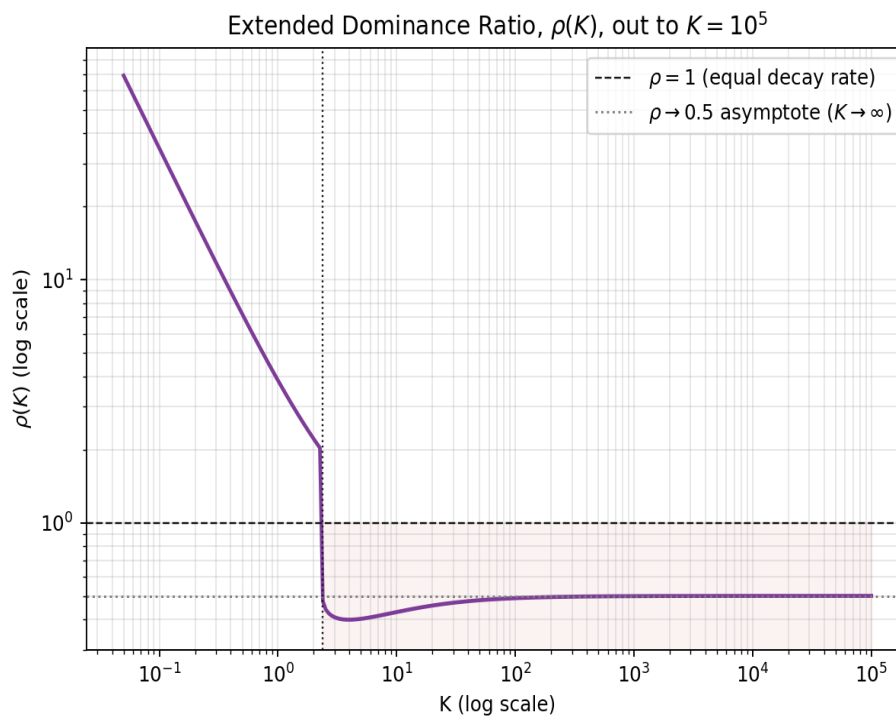


Figure 15. Dominance ratio $\rho(K)$ extended to $K = 10^5$ (log–log), showing the asymptotic approach to 0.5.

7.3 Residues: why the slow pole barely matters (until it briefly does)

Expanding the step response $Y(s) = T(s)/s$ in partial fractions gives a residue at every pole — its amplitude of participation in $y(t)$. At $K = 5$ the pole near $s = -0.2$ (the slowest-decaying one) carries a residue of only 0.149, smaller than either the other real pole's residue (0.414) or the dominant complex pair's (0.372 each); by $K = 20$ that same near-zero pole's residue has fallen to 0.026, and by $K = 100$ it is down to 0.005. In other words, as K grows the pole sliding toward $s = -0.2$ is approaching a near pole–zero cancellation with the fixed zero there, and a cancelling pole contributes an ever-smaller share of the response even though it is, strictly, the slowest one present.

Residue check ($K = 5$) – confirms the slow pole's small participation

```
from scipy.signal import residue
num_step = np.array([5*K, 6*K, K]) # numerator of T(s)
den_step = np.convolve(closed_loop_den(K), [1.0, 0.0]) # denominator of T(s)/s
r, p, _ = residue(num_step, den_step)
for ri, pi in zip(r, p):
    print(f'pole={pi:+.4f} | residue|={abs(ri):.5f}')
# K=5 -> pole=-0.2251 | residue|=0.149 (smallest of the four dynamic poles)
# K=20 -> pole=-0.2051 | residue|=0.026 (shrinking steadily as K grows)
```

This is the quantitative version of the qualitative claim in Section 6: the near-zero pole is too weakly excited to shape the peak (so PO and tp error can still fall to single digits at high K, Figure 7), but it decays slowly enough to leave a faint, stretched-out tail that briefly widens the 2%-band settling-time error around $K \approx 5$ before shrinking away as its residue keeps falling.

The code block above prints only two points ($K = 5$ and $K = 20$). Figure 16 turns this into a continuous trend from $K = 2.5$ to 300 (log-log), plotting the near-zero pole's residue magnitude alongside the dominant complex pair's. The contrast is stark: the near-zero pole's residue falls by nearly three orders of magnitude ($1.18 \rightarrow 0.0016$) across the swept range, close to a straight line on log-log axes — i.e. an approximate power law — while the dominant pair's residue stays within a factor of 1.7 of itself throughout ($0.29 \rightarrow 0.50$). The elevated value right at $K = 2.5$ is the same transition effect seen elsewhere: residues of individually-tracked poles inflate near the point where two poles are about to merge and split, since the partial-fraction decomposition is momentarily splitting one near-repeated root into two nearby ones.

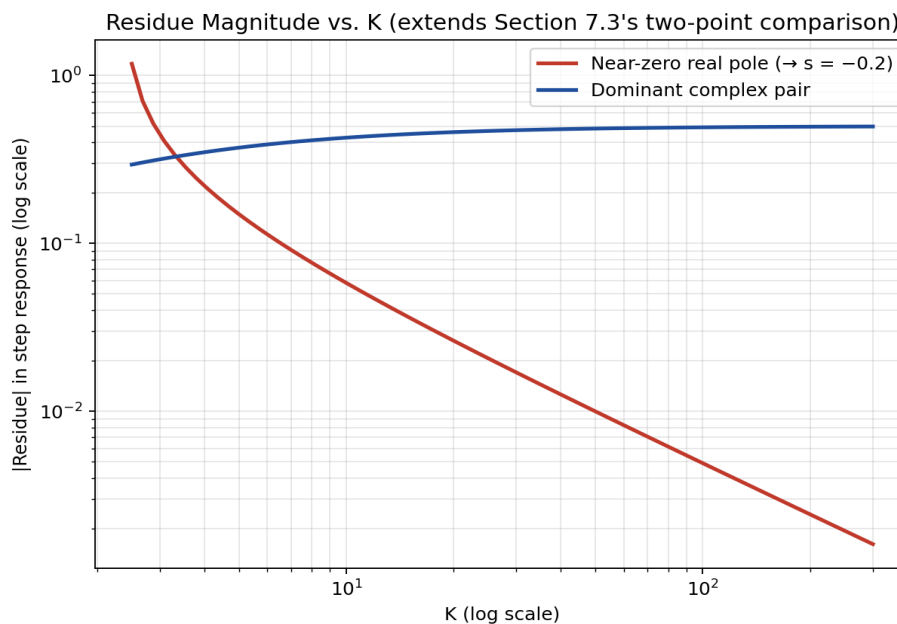


Figure 16. Residue magnitude of the near-zero real pole versus the dominant complex pair, $K = 2.5$ to 300 (log-log).

7.4 Type-2 verification via ramp and parabolic inputs

Section 3 argues $\text{ess} = 0$ for a step input from $T(0) = 1$, on the strength of the system being Type 2 (two free integrators in $G(s)$). That same Type-2 classification makes two further, sharper predictions for non-step inputs: zero steady-state error to a ramp (velocity error constant $K_v = \infty$ for any Type ≥ 1 system), and a finite, exactly-computable steady-state error to a parabolic input $r(t) = t^2/2$, via the acceleration error constant $K_a = \lim_{s \rightarrow 0} s^2 G(s) = K/10$, giving $\text{ess} = 1/K_a = 10/K$. Figure 19 simulates both directly at $K = 5$, where the prediction is $\text{ess} = 2.00$.

Type-2 Verification at $K = 5$ (supplementary / beyond brief)

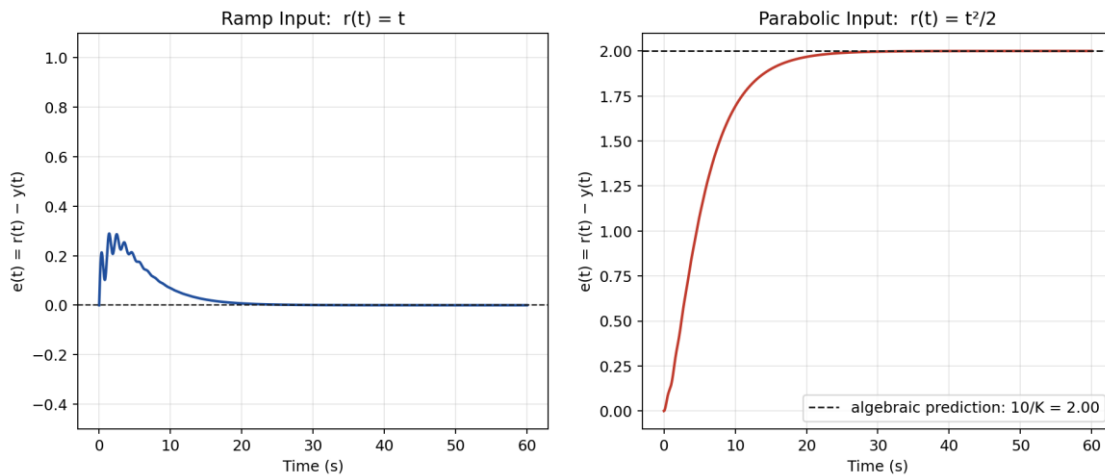


Figure 19. Error $e(t) = r(t) - y(t)$ at $K = 5$ for a ramp input $r(t) = t$ (left) and a parabolic input $r(t) = t^2/2$ (right).

The ramp error decays to zero, as predicted. The parabolic error converges to 1.99999 by $t = 60$ s against a predicted 2.00000 — agreement to four decimal places. This is a more direct demonstration of the Type-2 classification than the step-input argument alone: a step input cannot distinguish Type 1 from Type 2 (both give $\text{ess} = 0$), but the finite, non-zero, exactly- K -dependent parabolic error is a signature specific to Type 2. As with Section 7.4, this uses inputs beyond the step response the lab brief specifies — corroborating evidence, not required evidence.

7.6 Take-away for using dominant-pole reduction in general

The practical lesson generalises beyond this one transfer function: checking the real-part separation ratio ρ alone (Section 2.2's textbook rule of thumb) is necessary but not sufficient. It is equally important to check whether any zero — finite and fixed, exactly as in this system — sits close to the candidate dominant pole, and, where possible, to check the residue/participation of every other mode rather than only its decay rate. A pole can be simultaneously “slow” and “unimportant” if it is close to cancelling a nearby zero, which is precisely the situation this fourth-order system settles into at high gain.

Sections 7.4 and 7.5 add two independent cross-checks from outside the time domain and outside the step response — the frequency response and the ramp/parabolic error constants — and both agree with everything derived from the step response and pole locations alone. That agreement across independent methods is good evidence that the underlying analysis is correct, not just one particular measurement.

7.7 Cross-check with a second implementation (built-in functions, finer K -grid)

As a further check, the same experiment was re-run with a second, separately-written MATLAB script that reaches the same equations through different built-in commands: `zpk()` and `feedback()` build the closed-loop model instead of the polynomial construction of Section 4.1, and `stepinfo()` reads off the four metrics directly instead of the hand-written peak/settling-band code of Section 4.3. The second-order approximation is also built differently — rather than applying the PO/tp/ts formulas to ζ and ω_n directly, this script constructs an explicit unity-DC-gain transfer function $T_2 = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$

from the dominant pole and runs `stepinfo()` on that instead. The gain sweep is also considerably finer around the transition: $K = 0.1, 0.25, 0.5, 1, 1.5, 2, 2.5, 3, 4, 5, 7.5, 10, 20, 50, 100$, against the original eight.

Gain sweep and closed-loop model – built with `zpk()` and `feedback()` instead of a polynomial

```
K_values = [0.1 0.25 0.5 1 1.5 2 2.5 3 4 5 7.5 10 20 50 100];

for K = K_values
    G = zpk([-1 -0.2], [0 0 -1+3i -1-3i], 5*K); % same G(s) as Section 3, built from z/p/k
    T = feedback(G, 1);
    [PO, Tp, Ts, ess] = fourthOrderMetrics(T);
    [PO2, Tp2, Ts2, ess2, secondOrderPoles, T2] = secondOrderMetrics(T);
    % (row assembly and writetable() omitted – see Section 4.4)
end
```

Actual (fourth-order) step-response metrics, via `stepinfo()`

```
function [PO, Tp, Ts, ess] = fourthOrderMetrics(T)
    info = stepinfo(T);
    PO = info.Overshoot;
    Tp = info.PeakTime;
    Ts = info.SettlingTime;
    ess = abs(1 - dcgain(T));
end
```

Second-order approximation – built as an explicit transfer function, then measured with `stepinfo()`

```
function [PO, Tp, Ts, ess, p2, T2] = secondOrderMetrics(T)
    p = pole(T);
    p = p(imag(p) > 1e-8); % keep poles with positive imaginary part
    [~, i] = max(real(p)); % dominant = closest to imaginary axis
    pd = p(i);
    p2 = [pd; conj(pd)];

    sigma = -real(pd); wd = abs(imag(pd));
    wn = sqrt(sigma^2 + wd^2); zeta = sigma/wn;

    T2 = tf(wn^2, [1 2*zeta*wn wn^2]); % unity DC gain, matching T
    info = stepinfo(T2);
    PO = info.Overshoot; Tp = info.PeakTime;
    Ts = info.SettlingTime; ess = abs(1 - dcgain(T2));
end
```

Table 4 lists the resulting values across all fifteen gains.

K	OS meas. (%)	OS approx. (%)	OS err. (%)	tp meas. (s)	tp approx. (s)	tp err. (%)	ts meas. (s)	ts approx. (s)	ts err. (%)
0.1	44.81	39.21	12.5	27.96	32.90	17.6	135.59	139.73	3.1
0.25	30.54	21.45	29.8	16.47	22.90	39.0	50.23	55.46	10.4
0.5	20.79	9.67	53.5	11.32	19.31	70.5	31.06	29.12	6.3
1	13.02	2.01	84.6	7.89	18.98	140.5	17.94	19.32	7.7
1.5	9.70	0.23	97.6	7.03	22.80	224.1	15.54	15.21	2.1
2	8.49	0.00	100.0	5.22	26.47	407.5	13.90	16.61	19.5
2.5	11.88	63.85	437.7	0.75	0.68	9.1	12.60	5.65	55.2
3	19.23	66.60	246.4	0.71	0.64	9.1	11.51	5.97	48.1
4	30.09	70.78	135.2	0.64	0.58	9.1	10.19	6.55	35.7
5	38.40	73.82	92.2	0.54	0.54	0.0	9.38	6.63	29.3

K	OS meas. (%)	OS approx. (%)	OS err. (%)	tp meas. (s)	tp approx. (s)	tp err. (%)	ts meas. (s)	ts approx. (s)	ts err. (%)
7.5	51.74	78.74	52.2	0.46	0.46	0.0	8.86	7.46	15.7
10	59.76	81.73	36.8	0.41	0.41	0.0	8.69	7.85	9.7
20	74.52	87.33	17.2	0.30	0.30	0.0	8.79	8.48	3.5
50	86.27	92.11	6.8	0.20	0.20	0.0	9.21	9.20	0.1
100	91.34	94.44	3.4	0.14	0.14	0.0	9.35	9.48	1.4

Table 4. Measured versus second-order-approximate performance metrics from the second implementation, across fifteen gains.

The second pole pair (Pole3/Pole4 in the underlying data) is still complex at $K = 2$ ($-0.311 \pm j0.090$) and has split into two real poles by $K = 2.5$ (-0.401 and -0.290), bracketing the $K \approx 2.36$ transition located in Section 5.6 from the other side. With points on both sides of it, the transition's effect on the error columns comes through sharper than the original eight-gain table could show: overshoot error reaches 437.7% at $K = 2.5$, well past anything visible in Table 1, and settling-time error peaks at 55.2% at the same gain rather than as the smaller, apparently separate bump near $K = 5$ that Table 1 hinted at. It is the same feature in both tables — the extra points here just resolve where it actually sits, right at the transition rather than further out.

One number is worth flagging rather than quietly reconciling: at $K = 2$, this table reports a 407.5% peak-time error, against 576.0% at the same gain in Table 1. Both come from the same dominant pole ($\zeta \approx 0.96$, $\omega_d \approx 0.09$ rad/s) and the same underlying cause — ω_d is nearly zero there, so any peak-time estimate is on shaky ground — but the two scripts get different numbers because they compute tp_{approx} two different ways: Section 4.3 evaluates the closed-form $tp = \pi/\omega_d$ directly, while this script numerically locates the peak of a simulated T2 with `stepinfo()`. At ζ this close to 1 the predicted overshoot is a few thousandths of a percent, and finding the peak of an almost-flat curve numerically does not reliably reproduce the analytical π/ω_d value. Away from this one row the two scripts agree closely — tp itself matches to several decimal places for $K \geq 5$ — so this is best read as a caveat about locating a vanishing peak numerically, not a disagreement about the system.

Figure 20. Fourth-order step response for seven of the fifteen gains.

Figure 20 is the same kind of overlay as Figure 2, redrawn from this script's own simulation: the low- K curves ($K = 0.1$) settle only after several hundred seconds, while $K \geq 5$ settle within about ten.

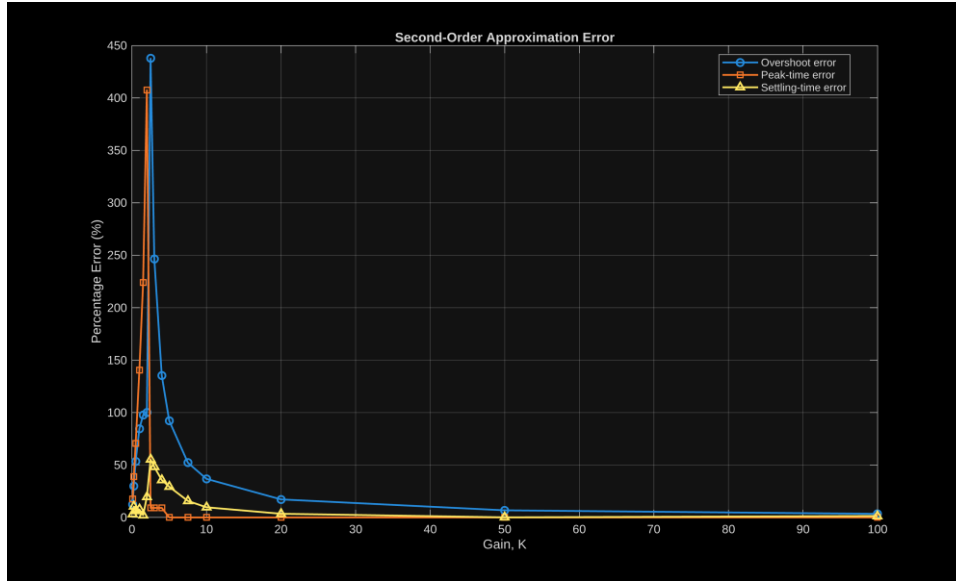


Figure 21. Overshoot, peak-time and settling-time error versus K , from Table 4.

8. Conclusion

The measured step response of this fourth-order, two-zero closed-loop system was compared against the standard second-order formulae across three orders of magnitude of gain. The two disagree sharply at low-to-moderate gain (PO error above 50% for $K \leq 0.5$, a 576% peak-time error right at the $K \approx 2.36$ transition) and converge to a much smaller, but persistent, single-digit-to-low-double-digit gap at high gain — a relationship visible both against K (Figure 8) and directly as measured-vs-approximate parity (Figure 6). Every one of these features was traced to a specific, verifiable cause rather than left as an empirical curve: two comparably-sized oscillatory modes and a nearby zero at low K ; a discontinuous relabelling of “dominant pole” exactly at the transition; and, at high K , a pole sliding toward a near pole–zero cancellation (Figure 16) that keeps the classical separation ratio below 1 forever — provably down to exactly 0.5 as $K \rightarrow \infty$, Figure 15 — while its shrinking residue keeps its practical effect small. The system remains stable and its steady-state error stays at or near zero across the entire tested range, independently confirmed in the frequency domain (Figures 17–18, infinite gain margin) and via ramp/parabolic inputs (Figure 19, exact $\text{ess} = 10/K$), not only from the step response. The $\pm 20\%$ open-loop pole-location study (Section 5.7) further shows that, even where the second-order approximation is at its best, the underlying performance metrics remain sensitive to parameter uncertainty: overshoot moved by roughly $+51\%/-47\%$ for only a $\pm 20\%$ shift in pole location. Taken together, this answers the objective in Section 1: a reduced-order model can be trusted only once both the separation and the zero-proximity conditions of Section 2.2 are checked — checking decay-rate separation alone, as the $K \geq 2.36$ region of this experiment demonstrates, is not enough.